Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3746027.3755425

RESEARCH-ARTICLE

From Model Diagram to Code: A Benchmark Dataset and Multi-Agent Framework

MENGZHEN WANG, South China University of Technology, Guangzhou, Guangdong, China

XUNBIN HUANG, South China University of Technology, Guangzhou, Guangdong, China

JIAYUAN XIE, The Hong Kong Polytechnic University, Hong Kong, Hong Kong, Hong Kong

SHUKAI MA, South China University of Technology, Guangzhou, Guangdong, China

JIALE MEN, South China University of Technology, Guangzhou, Guangdong, China

DAYONG LIANG, South China University of Technology, Guangzhou, Guangdong, China

View all

Open Access Support provided by:

South China University of Technology

The Hong Kong Polytechnic University

PDF Download
3746027.3755425.pdf
07 January 2026
Total Citations: 0
Total Downloads: 103

Published: 27 October 2025

Citation in BibTeX format

MM '25: The 33rd ACM International Conference on Multimedia
October 27 - 31, 2025
Dublin, Ireland

Conference Sponsors:
SIGMM

https://doi.org/10.1145/3746027.3755425
ISBN: 9798400720352

From Model Diagram to Code: A Benchmark Dataset and Multi-Agent Framework

Mengzhen Wang
South China University of Technology
Guangzhou, China
202311089439@mail.scut.edu.cn

Xunbin Huang
South China University of Technology
Guangzhou, China
202130490076@mail.scut.edu.cn

Jiayuan Xie
Hong Kong Polytechnic University
Hong Kong, China
jiayuan.xie@polyu.edu.hk

Shukai Ma
South China University of Technology
Guangzhou, China
202230481493@mail.scut.edu.cn

Jiale Men
South China University of Technology
Guangzhou, China
se202230481509@mail.scut.edu.cn

Dayong Liang
South China University of Technology
Guangzhou, China
ft_ldy@mail.scut.edu.cn

Yi Cai*
South China University of Technology
Guangzhou, China
Key Laboratory of Big Data and Intelligent Robot (SCUT), Ministry of Education
Guangzhou, China
ycai@scut.edu.cn

Abstract

Model Diagram-to-Code Generation aims to translate model diagrams from research papers into implementation code that reconstructs the model's architecture. This task plays a crucial role in accelerating scientific workflows and enhancing the efficiency of industrial model deployment. While recent studies have explored various Image-to-Code Generation tasks using Multimodal Large Language Models (MLLMs), these efforts have primarily focused on reconstructing the visual appearance depicted in input images, leaving this task largely underexplored. The complex structural elements and implicit relationships in model diagrams present greater challenges for MLLMs, particularly in terms of visual reasoning and semantic interpretation. To support this task, we introduce MDCDataset, a dataset designed to evaluate the ability of MLLMs to generate code from model diagrams. It comprises 1,008 instances spanning 16 research domains, each with a model diagram, structured textual content, and the ground-truth code implementation. Furthermore, to address the inherent challenges of this task, we propose MDCAgent, a collaborative multi-agent framework composed of Parsing, Generation, and Check Agents. These agents work in coordination to analyze, extract, and verify complex elements and implicit relationships within model diagrams, thereby enhancing the visual architecture-aware reasoning capabilities of MLLMs. Our extensive experiments confirm the effectiveness of the framework.[1]

CCS Concepts

• **Computing methodologies** → **Computer vision tasks**.

Keywords

Model Diagram-to-Code Generation; Multimodal Large Language Models; Multi-Agent Framework

ACM Reference Format:
Mengzhen Wang, Xunbin Huang, Jiayuan Xie, Shukai Ma, Jiale Men, Dayong Liang, and Yi Cai. 2025. From Model Diagram to Code: A Benchmark Dataset and Multi-Agent Framework. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25), October 27–31, 2025, Dublin, Ireland.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3746027.3755425

1 Introduction

Model Diagram-to-Code Generation aims to translate model diagrams from research papers into implementation code that reconstructs the model's architecture. This task is particularly relevant in computational research domains such as deep learning [9, 27].

A representative scenario is illustrated in Figure 1. When engaging with a research paper of interest, researchers often aim to access its underlying implementation. This helps them gain a deeper understanding of the proposed methodology or reuse specific components in their own work. However, many papers do not release source code, making model reproduction a necessary yet non-trivial task. This process can be particularly challenging for individuals who are unfamiliar with relevant implementation frameworks or have limited programming experience. In such cases, the model architecture diagram serves as a high-level abstraction of

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MM '25, Dublin, Ireland.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2035-2/2025/10
https://doi.org/10.1145/3746027.3755425

[1] Our dataset and code are available at https://github.com/afdyff/MDC.

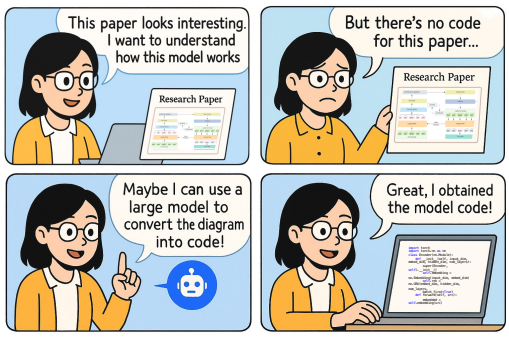


Figure 1: A real-world scenario in scientific research. This model diagram is reproduced from Zhao et al. [42].

the proposed method, capturing its modular structure and design logic. Given its value as a compact and informative representation, there is increasing interest in leveraging MLLMs [1, 4, 32] to automatically generate implementation code directly from model diagrams. This paradigm holds significant promise for accelerating research workflows. Similarly, in industrial settings, model development typically begins with high-level diagrams, such as model diagrams or flowcharts, which are then manually translated into executable code by engineers [16, 35]. This manual process is often time-consuming and error-prone. Enabling the direct conversion of diagrammatic representations into code can streamline the development pipeline and reduce engineering overhead.

Recent studies have increasingly investigated the performance of MLLMs on Image-to-Code generation tasks, which can be broadly classified into three categories: Scientific Chart Generation [12, 14, 29], UI-to-Code Translation [17, 30, 36], and Visual Programming [26, 34, 40]. The first two mainly focus on reconstructing the visual appearance depicted in input images, such as generating plotting scripts for scientific charts or producing HTML/CSS code for graphical user interfaces. Visual Programming addresses algorithmic problems, utilizing visual inputs (e.g., trees or graphs) as structural constraints to guide the generation of code. However, these tasks typically utilize input images composed of simple and clearly defined visual elements, making the visual reasoning process relatively straightforward [21, 28]. In contrast, model diagrams encode richer and more complex semantic information, including hierarchical modularity, mathematical symbols, and textual annotations. In addition, they often involve implicit relationships, such as connections between modules, execution order, and data flow, which are not visually explicit and must be inferred during code generation. These characteristics pose greater challenges for LLMs in understanding model diagrams for code generation [6–8, 43].

To the best of our knowledge, the performance of MLLMs on the Model Diagram-to-Code Generation remains underexplored in existing research. In this study, we aim to bridge this research gap. We first construct a high-quality benchmark dataset, **MDCDataset**, which consists of 1,008 instances collected from research papers published between 2015 and 2025, spanning 16 diverse research domains. Each instance contains: (1) a model diagram extracted, (2) its associated structured textual content (e.g., Abstract, Method) from the corresponding paper, and (3) the ground-truth code implementation of the model architecture. Based on this dataset, we conduct extensive experiments to evaluate the ability of state-of-the-art MLLMs to translate model diagrams into architectural code. We also investigate their capacity for cross-modal understanding—specifically, whether they can effectively leverage relevant textual information from the paper to support code generation.

Furthermore, to address the inherent challenges of this task, we propose a collaborative multi-agent framework, **MDCAgent**, which comprises three specialized agents: the *Parsing Agent*, the *Generation Agent*, and the *Check Agent*. The *Parsing Agent* is responsible for analyzing the model diagram to identify complex architectural elements and extract implicit relationships, thereby determining the inter-module execution order and component-level dependencies. It then produces a detailed analysis of the model architecture, which serves as structured input for the *Generation Agent*. Guided by this parsed information, the *Generation Agent* translates the model diagram into code. Finally, the *Check Agent* verifies and refines the generated code by comparing it against both the diagram and the parsed information, thereby ensuring the completeness of architectural elements and the correctness of inferred relationships. This framework emulates the divide-and-conquer strategy commonly employed in human software engineering workflows, leveraging the complementary capabilities of modular agents to improve the overall quality and reliability of code generation. In summary, our contributions are as follows:

- We construct **MDCDataset**, a dataset for the Model Diagram-to-Code Generation task, consisting of 1,008 high-quality triplets of model diagrams, structured text, and ground-truth code implementations. This dataset fills a gap in current research and provides a benchmark for future studies.
- We propose **MDCAgent**, a collaborative multi-agent framework designed to address the challenges of the Model Diagram-to-Code Generation task. By coordinating three specialized agents, it enhances MLLMs' architecture-aware reasoning.
- Extensive experimental results demonstrate that MDCAgent enhances the performance of MLLMs and can serve as a baseline for future research.

2 Related Work

2.1 Image-to-Code Generation

Image-to-Code Generation aims to develop models capable of generating code from images. This work spans various tasks, including Visual Programming [10, 34, 40], UI-to-Code Translation [17, 36], and Scientific Chart Understanding and Generation [12, 14, 21]. In the field of Visual Programming, Li et al. [20] propose MMCode, a multimodal dataset designed to evaluate the algorithmic problem-solving abilities of MLLMs in visually rich scenarios. To further assess their reasoning capabilities, OlympiadBench [13] introduces challenging bilingual multimodal math and physics problems at the Olympiad level. For Scientific Chart Generation, Zhang et al. [41] investigate the potential of MLLMs to automate data visualization by generating code templates from existing charts. Wu et al. [38] introduce a compact dataset of 132 matplotlib code–plot pairs from general domains, enabling an in-depth evaluation of MLLMs' performance. In UI-to-Code Translation, Roberts et al. [28] propose

Image2Struct, a benchmark for evaluating MLLMs in extracting structured representations (e.g., LaTeX or HTML) from images such as webpage screenshots. Similarly, Soselia et al. [31] construct two synthetic datasets comprising unique (code, screenshot) pairs to train MLLMs for generating HTML/CSS from UI images.

While significant progress has been made in enabling MLLMs to generate code from visual elements, existing research has not addressed the challenge of generating code directly from model diagrams. This work aims to fill this gap by exploring the ability of MLLMs to learn this new form of multimodal understanding.

2.2 Multimodal Large Language Models

MLLMs have recently gained attention for their ability to integrate multiple modalities to perform complex tasks. Proprietary MLLMs, such as GPT-4V [1], Gemini [32], and Claude3 [4], have demonstrated advanced capabilities in enabling complex multimodal interactions. Similarly, open-weight LMMs, such as LLaVA [19], Qwen2-VL [37], and DeepSeek-VL [24], have showcased strong cross-modal integration and improved generalization. Several works[20, 31, 41] have already evaluated the multimodal understanding and analysis capabilities of MLLMs in different tasks and settings, particularly focusing on Image-to-Text and Image-to-Code Generation. These studies typically involve visual elements such as web screenshots, PDFs, OCR text, and charts. However, compared to the relatively straightforward visual elements commonly studied, model diagrams often contain intricate structures, symbolic elements, and complex relationships that require advanced reasoning for full comprehension. These complexities pose significant challenges for MLLMs, which must not only interpret visual content but also understand the underlying context and dependencies to generate functional code. To this end, we conduct a comprehensive evaluation of state-of-the-art MLLMs on this task, and further propose a collaborative multi-agent framework to enhance their ability to understand complex elements and relationships within model diagrams.

3 Dataset

The construction of **MDCDataset** was undertaken by a team comprising two Ph.D. candidates and four M.S. students in computer science, all possessing extensive programming experience. As shown in Figure 2, the construction process consists of two main stages: (1) Data Collection and (2) Data Parsing.

3.1 Data Collection

PapersWithCode[2] is an established platform that provides curated collections of state-of-the-art research papers in the field of machine learning, along with associated code implementations links. It serves as a valuable resource for identifying high-quality research, as the papers listed on this platform are typically validated through peer and community assessments, ensuring a higher standard of reliability. For this study, we utilized the PapersWithCode API[3] to collect 1,600 arXiv IDs and their corresponding code implementation links. Subsequently, we obtained the LaTeX source files from arXiv[4] based on the arXiv IDs, and retrieved the source code files from GitHub[5] based on the code implementation links.

3.2 Data Parsing

As shown in the lower block of Figure 2, we parse both the LaTeX source files and the source code files to extract three key components: (1) the model architecture diagram, (2) the structured textual content, and (3) the ground-truth code implementation.

Model Architecture Diagram. The LaTeX source files of research papers typically contain all figure resources used in the manuscript. Given that our study specifically focuses on model architecture diagrams, we aim to extract only those images that directly represent the structural design of the proposed models. To this end, we develop a hybrid approach that combines rule-based filename matching with manual verification. In the initial stage, we perform a coarse-grained filtering of figure files based on heuristic keywords present in their filenames (e.g., "model", "architecture"). This yields a candidate set of images that are likely to depict model architectures. However, due to variations in naming conventions across authors, this rule-based approach may inadvertently include irrelevant figures, such as visualizations of results, data preprocessing pipelines. To enhance the precision of extraction, we further incorporate a manual verification step. Specifically, each candidate image is checked against its corresponding mention or citation in the main LaTeX source file, which contains the complete content of the paper and is typically the central .tex file (hereafter referred to as *main.tex*) used to compile the final manuscript. Through this process, we not only eliminate unrelated images but also ensure that the retained figures contain sufficient structural information to accurately reflect the model's core architecture.

Subsequently, the diagrams are categorized into three levels of complexity. We first employ an MLLM to analyze their structural characteristics, taking into account multiple factors such as the number of functional modules, connectivity patterns, the presence of hierarchical structures, and the use of symbolic or mathematical components. Based on these criteria, each diagram is initially assigned a difficulty level: *easy*, *medium*, or *hard*. To ensure the reliability of this classification, human annotators further review and refine the MLLM-generated labels through manual inspection, taking into account both visual and semantic aspects of the diagrams. In this scheme, diagrams classified as *easy* typically contain fewer components with simple linear or sequential connections. In contrast, *medium* diagrams involve moderate modular granularity and more intricate interconnections, while *hard* diagrams exhibit highly complex topologies, often featuring nested modules, multi-branch flows, or hybrid architectures.

Structured Textual Content. To evaluate the ability of MLLMs to leverage textual information when generating code from model architecture diagrams, we further extract the *main.tex* file to obtain the section-level textual content of the paper. However, academic papers vary significantly in their structural formatting due to differences in conference and journal templates, as well as individual author styles. Section divisions are often inconsistent across papers: for example, some papers merge the "Method" and "Experiments"

[2]https://paperswithcode.com/sota
[3]https://paperswithcode-client.readthedocs.io/
[4]https://info.arxiv.org/help/api/basics.html
[5]https://github.com/

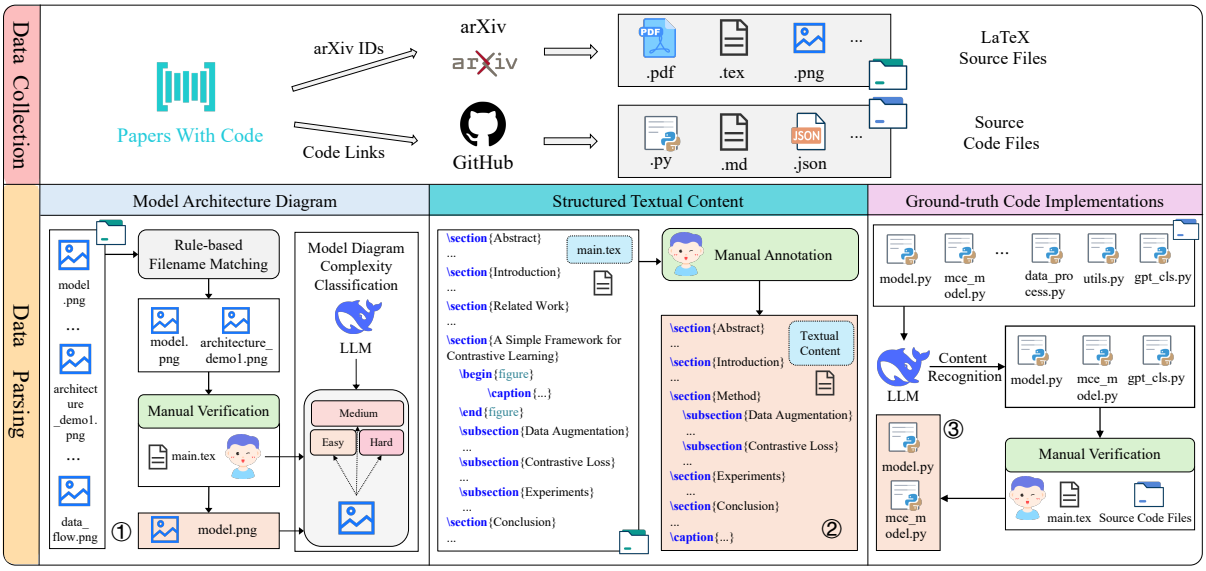


Figure 2: The construction pipeline of MDCDataset. The highlighted orange box denotes the three final components extracted for each data instance: ① Model Architecture Diagram, ② Structured Textual Content, and ③ Ground-truth Code Implementations.

sections into one. Additionally, non-standard section titles such as "Approach," or "Model" are frequently used to refer to the methodology. To address this variability, we establish a unified annotation scheme and manually classify the textual content of each paper into five standardized sections: Abstract, Introduction, Method, Experiments, and Conclusion. Specifically, human annotators identify section headers from `\section` commands in the *main.tex* file and assign standardized labels based on paragraph semantics. In addition, we recognize that figure captions accompanying model architecture diagrams often contain concise and informative descriptions of the model—such as component functions, flow logic, and architectural highlights. Therefore, we manually extract the captions corresponding to the selected diagrams and include them as part of the textual context used for model evaluation. Furthermore, we incorporate the full paper content as a source of context for evaluating the capabilities of MLLMs.

Ground-truth Code Implementation. We obtain the official code released by the original authors through provided links and treat it as the ground-truth code implementation, which is both credible and reproducible. Additionally, we restrict our selection to Python code to avoid structural deviations that may arise from non-standard or third-party reproductions. GitHub repositories for paper implementations often include various files such as .py scripts for model structures, training, and evaluation, as well as dependency files (e.g., requirements.txt) and configuration or execution scripts. In this study, our focus is exclusively on the implementation of model architecture functionality. Accordingly, we concentrate on .py files and extract class and function definitions that are directly responsible for constructing the model structure. To achieve this, we adopt a two-stage extraction pipeline that combines LLM assistance with manual verification.

In the first stage, all .py files in the source code are analyzed by the LLM, which identifies candidate files likely to contain model architecture definitions based on structural patterns and content semantics. In the second stage, human annotators manually review the candidate files by comparing them with the original paper and the source code repository. This review determines whether the selected files sufficiently and accurately represent the core architecture of the model. If essential components are missing, additional files from the repository are incorporated to complete the architectural definition. Conversely, files unrelated to the model structure, such as evaluation routines or data preprocessing utilities, are excluded to maintain a focused and accurate representation.

Following the data parsing process described above, we construct the **MDCDataset**, which comprises 1,008 high-quality instances. Detailed dataset analysis is provided in the supplementary material.

4 Task Definition

In this section, we define the **Model Diagram-to-Code Generation** task. Given a model architecture diagram D and a general-purpose instruction prompt I, the objective is to enable an MLLM f to generate code C. Formally, the task can be defined as:

$$C = f(D, I) \tag{1}$$

To further assess MLLMs' ability to leverage textual architectural information for code generation, we extend the task by incorporating auxiliary textual content extracted from pertinent sections (e.g., *Introduction*, *Method*) of the source paper. In this enhanced setting, the input includes a model architecture diagram D, additional textual context T, and an instruction prompt I. The goal remains to generate the implementation code C, defined as:

$$C = f(D, T, I) \tag{2}$$

It is worth noting that a single model architecture may admit multiple valid implementation variants, which can affect the accuracy of automatic evaluation metrics. To ensure evaluation reliability,

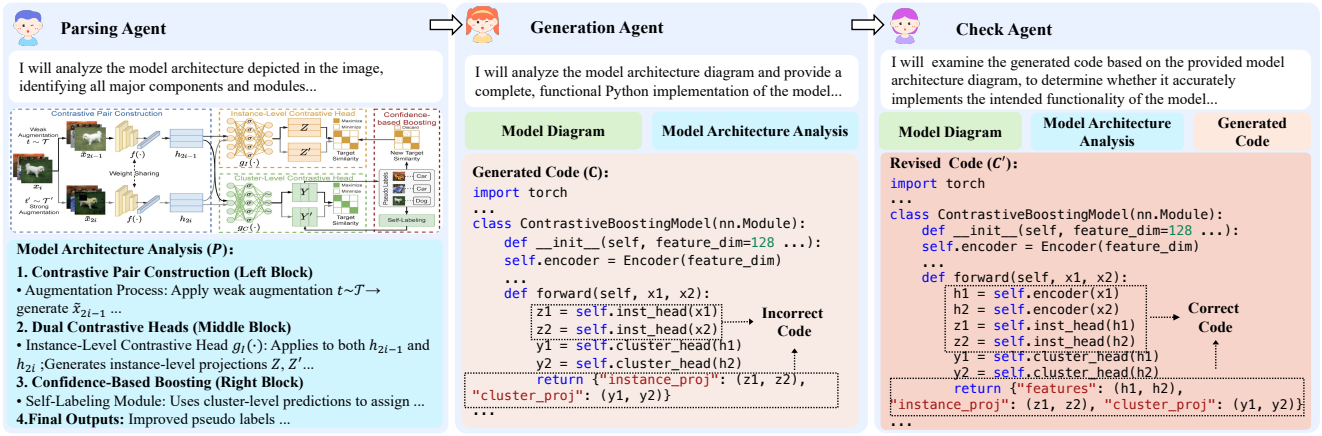


Figure 3: Overview of the MDCAgent Framework. This model diagram is reproduced from Li et al. [22].

we follow the common practice in code generation tasks [8, 15, 43], which focuses on generating the functional body of the code given a predefined function signature. Specifically, we provide the model with imported packages and class definitions from the ground-truth implementation as part of the instruction I, and require the MLLM to complete the internal functionality accordingly.

5 Method

As shown in Figure 3, the **MDCAgent** consists of three specialized agents: the *Parsing Agent*, *Generation Agent*, and *Check Agent*. Each agent plays a distinct functional role within the overall pipeline, following a divide-and-conquer strategy inspired by human approaches to software design and implementation. The prompts used for each agent are provided in the supplementary material.

5.1 Parsing Agent

The Parsing Agent is responsible for structural analysis and dependency extraction based on the model diagram D. It first parses the diagram to identify key architectural elements, including modules, symbols, and associated textual annotations. Subsequently, it extracts implicit structural relationships, such as intermodule associations and data flow dependencies, to infer the execution order and component-level interactions. This process results in a structured analysis P that captures both the architectural components and their logical organization, serving as input for the Generation Agent. Formally, the parsing process can be defined as:

$$P = f_{\text{parse}}(D, I_{\text{parse}}) \tag{3}$$

where f_{parse} denotes the parsing function powered by an MLLM, and I_{parse} is a task-specific parsing instruction prompt.

5.2 Generation Agent

The Generation Agent takes the architectural analysis P and the original diagram D as input and generates the initial version of the code C. This agent focuses on translating high-level structural semantics into code that adheres to deep learning framework conventions. The generation process is defined as:

$$C = f_{\text{gen}}(D, P, I_{\text{gen}}) \tag{4}$$

where f_{gen} denotes the code generation function, also implemented using an MLLM. I_{gen} is a generation-specific instruction prompt.

5.3 Check Agent

The Check Agent serves as a verifier and refiner. It analyzes the generated code C in relation to the original diagram D and the analysis P to identify missing modules, incorrect logic, or incomplete coverage. If any inconsistencies are detected, the agent updates C to produce a corrected version C':

$$C' = f_{\text{check}}(D, P, C, I_{\text{check}}) \tag{5}$$

where f_{check} denotes the verification and refinement function powered by an MLLM. I_{check} is a check-specific instruction prompt. If C is already complete, $C' = C$; otherwise, the Check Agent directly edits the code to improve its completeness and correctness.

6 Experiments

6.1 Evaluation Metrics

We evaluate generated code against ground-truth implementations using both LLM-based scoring and automatic metrics: LLM Score (↑), CodeBLEU (↑), CrystalBLEU (↑), ROUGE-L (↑), chrF (↑), Edit Distance (↓). Additionally, we manually assess a subset of the data to ensure evaluation reliability. More details about these evaluation metrics are provided in the supplementary material.

6.2 Baseline Setup

We benchmark 16 representative MLLMs, including both proprietary and open-weight models that are widely adopted.

Open-weight Models. We evaluate 10 competitive open-weight models, whose checkpoints are publicly released. These models include: DeepSeek-VL-7B [24], Qwen-VL-Plus [37], Qwen2.5-VL-72B [5], Yi-VL-6B [39], LLaVA-v1.5-7B [23], Mistral-small-3.1-24B-Instruct [2], Gemma-3-27B-it [33], GLM-4V-9B [11], MiniMax-VL-01 [18], and LLaMA-3.2-11B-Vision-Instruct [25].

Proprietary Models. We consider 6 high-performing proprietary models that are publicly accessible via APIs. These include: GPT-4o [1], GPT-4V [1], Claude-3.5-Haiku [4], Claude-3.7-Sonnet [4], Gemini-2.0-Flash-Lite [32], and Nova-Lite-1.0 [3].

Table 1: Performance of MLLMs on the Model Diagram-to-Code Generation task. Best scores are shown in bold, with second-best scores underlined.

Type	Model	Size	LLM Score↑	CodeBLEU↑	CrystalBLEU↑	ROUGE-L↑	chrF↑	Edit Distance↓
Open weight	DeepSeek-VL	7B	9.31	11.94	6.91	6.22	11.94	85.93
	Qwen-VL-Plus	-	34.91	19.54	12.99	23.74	24.38	78.81
	Qwen2.5-VL	72B	49.29	27.50	22.53	34.43	35.63	70.61
	Yi-VL	6B	7.28	8.20	4.32	5.59	5.34	89.10
	LLaVA-V1.5	7B	7.51	9.93	5.31	6.46	6.23	90.14
	Mistral-small-3.1-Instruct	24B	49.74	29.82	32.19	35.18	40.92	67.74
	Gemma-3-it	27B	40.62	23.70	19.24	29.31	31.22	73.90
	GLM-4V	9B	14.59	14.68	7.50	9.49	13.52	86.19
	MiniMax-VL-01	456B	48.53	26.88	24.57	32.03	35.48	71.07
	LLaMA-3.2-Vision-Instruct	11B	29.81	20.27	18.81	19.88	27.06	79.31
Proprietary	GPT-4o	-	50.17	26.39	23.54	32.91	35.68	71.09
	GPT-4V	-	38.81	21.52	12.95	26.70	28.07	77.54
	Claude-3.5-Haiku	-	50.51	27.05	30.99	30.65	37.98	71.51
	Claude-3.7-Sonnet	-	**56.14**	**34.52**	**46.59**	35.86	**49.79**	**66.10**
	Gemini-2.0-Flash-Lite	-	50.84	30.34	36.57	**36.07**	42.63	66.63
	Nova-Lite-1.0	-	42.02	24.55	21.97	29.88	33.42	72.87

Table 2: Performance of MLLMs on the Model Diagram-to-Code Generation task with textual context. Scores are averaged over six metrics, including (1 - Edit Distance) and five other standard metrics. Results with performance gains and ranking in the top three are highlighted in bold. "#" indicates results without textual context.

Model	Size	#	Caption	Abstract	Introduction	Method	Experiment	Conclusion	Full Paper Content
LLaMA-3.2-Vision-Instruct	11B	22.75	22.36 (-0.39)	**23.22 (+0.47)**	22.01 (-0.74)	21.32 (-1.43)	20.39 (-2.36)	21.07 (-1.68)	16.40 (-6.35)
Gemma-3-it	27B	28.37	27.50 (-0.87)	28.97 (+0.60)	**29.75 (+1.38)**	**30.23 (+1.86)**	**29.94 (+1.57)**	28.00 (-0.37)	28.73 (+0.36)
Qwen2.5-VL	72B	33.13	33.47 (+0.34)	33.16 (+0.03)	**34.61 (+1.48)**	**33.71 (+0.58)**	33.67 (+0.54)	33.20 (+0.07)	**34.80 (+1.67)**
GPT-4o	-	32.93	32.30 (-0.63)	33.82 (+0.89)	**33.96 (+1.03)**	**34.67 (+1.74)**	**34.17 (+1.24)**	31.85 (-1.08)	33.93 (+1.00)
Claude-3.7-Sonnet	-	42.80	44.37 (+1.57)	44.83 **(+2.03)**	**45.38 (+2.58)**	**46.46 (+3.66)**	45.36 (+2.56)	42.94 (+0.14)	**46.60 (+3.8)**
Gemini-2.0-Flash-Lite	-	38.30	39.98 (+1.68)	40.26 (+1.96)	**41.13 (+2.83)**	**41.32 (+3.02)**	40.60 (+2.30)	39.85 (+1.55)	**41.61 (+3.31)**

6.3 Implementation Details

For all experiments, we adopt a unified prompt for both models to ensure fair and consistent evaluation. Each model uses its maximum input and output length to support long-form multimodal reasoning and code generation. The temperature is fixed at 0.3 to reduce randomness while preserving diversity. All models are run either via official APIs with default configurations or locally on two A100 GPUs, depending on deployment availability. DeepSeek-V3 is utilized to support the construction process of MDCDataset.

6.4 Performance of MLLMs on Model Diagram-to-Code Generation

As shown in Table 1, we conduct extensive experiments to evaluate the capability of MLLMs to understand model diagrams and generate code. The evaluation reveals a clear performance gap between proprietary and open-weight MLLMs. Proprietary models such as Claude-3.7-Sonnet and Gemini-2.0-Flash-Lite consistently outperform open-weight models across all evaluation metrics. While larger open-weight models like Qwen2.5-VL and MiniMax-VL-01 demonstrate some promising results, smaller ones such as Yi-VL and LLaVA-V1.5 perform significantly worse, highlighting their limitations in interpreting complex scientific diagrams. This disparity may be attributed to differences in model size, the scale of multimodal training data, and the fine-tuning strategies adopted by commercial systems.

Although top proprietary models demonstrate relative advantages, all MLLMs still fall short of optimal performance on this task. One possible reason lies in the training objectives of MLLMs, which primarily focus on tasks such as image content recognition and visual question answering. These tasks emphasize content-level understanding rather than structural reasoning, and lack the capability to map complex visual layouts to code. As a result, MLLMs are ill-equipped to capture the implicit relationships required for accurate diagram-to-code translation.

6.5 Performance of MLLMs on Model Diagram-to-Code Generation with Textual Context

This section examines how textual context from different sections of papers affects MLLM performance on the Model Diagram-to-Code Generation task. Table 2 reports results for six representative MLLMs across various context settings, including caption, abstract, introduction, method, experiment, conclusion, and the full paper.

Experimental results show that the method section consistently yields the most significant performance gains. This suggests that its rich structural descriptions and implementation details offer

Table 3: Performance comparison between MDCAgent and the baselines. "(w/ Content)" indicates the model's average performance when provided with textual context from different sections of the paper. "Avg." denotes the overall average performance.

Model	Size	LLM Score↑	CodeBLEU↑	CrystalBLEU↑	ROUGE-L↑	chrF↑	Edit Distance↓
LLaMA-3.2-Vision-Instruct	11B	29.81	20.27	18.81	19.88	27.06	79.31
MDCAgent	11B	32.51 ↑	22.62 ↑	22.13 ↑	22.92 ↑	26.69 ↓	77.12 ↓
LLaMA-3.2-Vision-Instruct (w/ Content)	11B	28.63	18.91	16.21	17.87	24.36	80.16
MDCAgent (w/ Content)	11B	29.71 ↑	20.18 ↑	17.23 ↑	18.49 ↑	24.21 ↓	78.14 ↓
Gemma-3-it	27B	40.62	23.70	19.24	29.31	31.22	73.90
MDCAgent	27B	43.77 ↑	23.55 ↓	22.26 ↑	33.29 ↑	32.17 ↑	71.30 ↓
Gemma-3-it (w/ Content)	27B	40.68	24.36	20.31	29.93	31.89	73.06
MDCAgent (w/ Content)	27B	43.02 ↑	25.89 ↑	22.78 ↑	32.13 ↑	32.00 ↑	71.32 ↓
Qwen2.5-VL	72B	49.29	27.50	22.53	34.43	35.63	70.61
MDCAgent	72B	53.01 ↑	29.61 ↑	25.77 ↑	37.56 ↑	37.86 ↑	68.01 ↓
Qwen2.5-VL (w/ Content)	72B	50.92	27.46	23.53	34.80	36.71	70.16
MDCAgent (w/ Content)	72B	53.24 ↑	29.60 ↑	27.08 ↑	38.15 ↑	38.00 ↑	68.49↓
GPT-4o	-	51.17	26.39	23.54	32.91	35.68	71.09
MDCAgent	-	54.29 ↑	28.67 ↑	25.69 ↑	32.23 ↓	37.80 ↑	67.99 ↓
GPT-4o (w/ Content)	-	51.57	26.59	24.78	32.74	36.15	70.67
MDCAgent (w/ Content)	-	53.87 ↑	29.01 ↑	26.18 ↑	33.85 ↑	38.50 ↑	66.45 ↓
Claude-3.7-Sonnet	-	56.14	34.52	46.59	35.86	49.79	66.10
MDCAgent	-	**59.32** ↑	**36.77** ↑	**50.13** ↑	35.81 ↓	**51.72** ↑	**65.11** ↓
Claude-3.7-Sonnet (w/ Content)	-	58.38	36.86	50.39	37.07	52.85	64.73
MDCAgent (w/ Content)	-	**60.53** ↑	**39.89** ↑	**52.41** ↑	**39.12** ↑	**55.41** ↑	**62.08** ↓
Gemini-2.0-Flash-Lite	-	50.84	30.34	36.57	**36.07**	42.63	66.63
MDCAgent	-	53.63 ↑	33.55 ↑	36.76 ↑	39.10 ↑	44.39 ↑	65.87 ↓
Gemini-2.0-Flash-Lite (w/ Content)	-	53.07	32.82	38.71	38.15	45.80	64.48
MDCAgent (w/ Content)	-	55.49 ↑	35.91 ↑	41.22 ↑	40.00 ↑	47.24 ↑	62.76 ↓
All Models Avg.	-	46.32	27.12	27.88	31.41	37.00	71.27
MDCAgent Avg.	-	49.42 (+3.10)	29.13 (+2.01)	30.46 (+2.58)	33.49 (+2.08)	38.44 (+1.44)	69.23 (-2.04)
All Models Avg. (w/ Content)	-	47.21	27.83	28.99	31.76	37.96	70.54
MDCAgent Avg. (w/ Content)	-	49.31 (+2.10)	30.08 (+2.25)	31.15 (+2.16)	33.62 (+1.86)	39.23 (+1.27)	68.20 (-2.34)

explicit guidance for MLLMs to understand model diagrams. The introduction and full paper content also lead to notable improvements: the former provides high-level overviews that aid model comprehension, while the latter offers comprehensive context that benefits models with strong long-context capabilities. However, the effectiveness of full-text input depends on the model's ability to identify relevant information, which may be compromised by overly long context introducing distraction or noise. In contrast, context from the caption, abstract, and conclusion yields limited improvement, likely due to their lack of technical specificity and structural detail. More detailed experimental results are provided in the supplementary material.

6.6 Performance of the MDCAgent

We apply MDCAgent to six MLLMs and evaluate its effectiveness on the Model Diagram-to-Code Generation task, both with and without textual context. As shown in Table 3, the results demonstrate that MDCAgent consistently improves generation performance. Without textual context, the average LLM Score increases by more than 3 points, and all other metrics—except chrF—improve by over 2 points. With textual context, MDCAgent still delivers clear gains, achieving over 2 points on LLM Score, CodeBLEU, CrystalBLEU, and Edit Distance. Moreover, we observe that MDCAgent without textual context still outperforms the baselines with textual context. These results demonstrate the robustness of MDCAgent.

These performance gains can be primarily attributed to the design of MDCAgent, which introduces structured parsing and verification into the generation process. Compared to relying on lengthy textual context to locate structural information, the *Parsing Agent* offers a more direct and diagram-grounded approach to extracting architecture-relevant semantics. It identifies complex architectural elements and extracts implicit relationships to determine inter-module execution order and component-level dependencies, thereby guiding the *Generation Agent* to produce code with stronger architectural alignment. Moreover, the *Check Agent* effectively mitigates common errors such as module omission or incorrect connections, ensuring greater code completeness. These findings suggest that MDCAgent enhances the structural reasoning of MLLMs in complex multimodal settings.

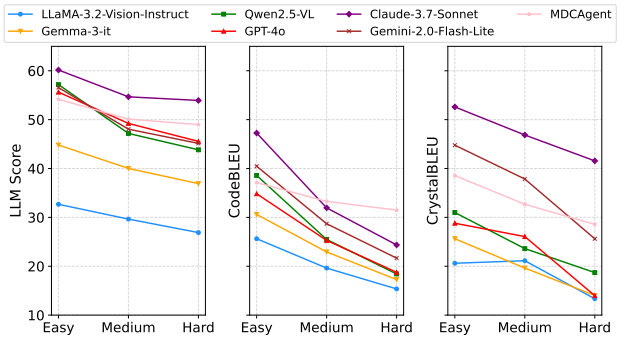


Figure 4: Performance of MLLMs across complexity levels.

6.7 Different Complexity Levels

To further investigate how model diagram complexity affects MLLM performance, we analyze the results of seven MLLMs on the Model Diagram-to-Code Generation task across three complexity levels:

Table 4: Ablation study showing average performance across six models. "w/o Parsing" denotes MDCAgent without the *Parsing Agent*, and "w/o Check" denotes removal of the *Check Agent*.

Task	Model	LLM Score↑	CodeBLEU↑	CrystalBLEU↑	ROUGE-L↑	chrF↑	Edit Distance↓
Model Diagram-to-Code	MDCAgent	49.42	29.13	30.46	33.49	38.44	69.23
	MDCAgent (w/o Parsing)	47.30 (-2.12)	27.55 (-1.58)	28.84 (-1.62)	31.60 (-1.89)	37.35 (-1.09)	70.47 (+1.24)
	MDCAgent (w/o Check)	47.44 (-1.98)	27.81 (-1.32)	28.78 (-1.68)	31.55 (-1.94)	37.41 (-1.03)	70.76 (+1.53)
Model Diagram-to-Code (w/ Content)	MDCAgent	49.31	30.08	31.15	33.62	39.23	68.20
	MDCAgent (w/o Parsing)	47.75 (-1.56)	28.46 (-1.62)	29.65 (-1.50)	32.22 (-1.40)	38.34 (-0.89)	70.30 (+2.10)
	MDCAgent (w/o Check)	47.94 (-1.37)	28.91 (-1.17)	30.01 (-1.14)	32.56 (-1.06)	38.40 (-0.83)	70.15 (+1.95)

Table 5: Correlation between automatic evaluation metrics and human judgments.

Correlation Type	LLM Score	Code Metrics	Text Metrics
Pearson Correlation	0.872	0.858	0.810
Spearman Rank Correlation	0.881	0.867	0.781

easy, medium, and hard. As shown in Figure 4, we report performance on three metrics: LLM Score, CodeBLEU, and CrystalBLEU. The results reveal a clear trend: model performance consistently declines as diagram complexity increases. Proprietary models like Claude-3.7-Sonnet perform well on easy cases and remain relatively robust on more complex ones. In contrast, open-weight models such as LLaMA-3.2-Vision-Instruct and Gemma-3-it not only lag behind overall but also show sharper performance drops. Notably, the decline from easy to medium complexity is generally moderate, while a more significant drop occurs from medium to hard. These results suggest that increasing structural complexity and semantic density present significant challenges for existing models.

6.8 Ablation Study

We conduct an ablation study to examine the contribution of each component within MDCAgent. As shown in Table 4, removing the *Parsing Agent* results in the most significant performance decline. The LLM Score drops by 1.56 to 2.12 points, while CodeBLEU, CrystalBLEU, and ROUGE-L decrease by approximately 1.4 to 1.89 points. In addition, Edit Distance increases by 1.24 to 2.10, indicating a higher degree of structural mismatch. This highlights the *Parsing Agent*'s critical role in structured reasoning and guiding code generation. The *Check Agent* also proves essential. Its removal leads to a decline of 1.37 to 1.98 points in LLM Score, along with a reduction of 1.06 to 1.94 points in CodeBLEU, CrystalBLEU, and ROUGE-L. Edit Distance increases by 1.53 to 1.95, suggesting that the Check Agent plays a key role in refining the generated code and ensuring its consistency with the input diagram. Details on model errors and failure cases are in the supplementary material.

6.9 Correlation with Human Evaluation

To assess the reliability of automatic evaluation metrics, we conduct a human evaluation. Specifically, we randomly select 50 samples from each complexity level, resulting in a total of 150 instances. For each instance, the code generated by six MLLMs (the same models used in Section 6.5) is compared against the ground truth using three evaluation criteria. The first is module implementation completeness, which checks whether the generated code includes all necessary modules without omissions or redundancies. The second is functional implementation accuracy, which evaluates whether the internal logic or functionality of each module is correctly implemented in the generated code. The third is module linkage correctness, which assesses whether the inter-module relationships and calling structures match those depicted in the original diagram. Each criterion is independently scored on a scale of 0 to 100 by human annotators.

We measure the alignment between human judgment and three groups of automatic metrics using Pearson and Spearman correlation coefficients. Specifically, the Code Metrics group includes CodeBLEU and CrystalBLEU, while the Text Metrics group comprises ROUGE-L, chrF, and Edit Distance. To facilitate comparison, we compute the average scores across the six MLLMs for each metric group and correlate them with human ratings. As shown in Table 5, all metric categories exhibit positive correlations with human scores, indicating their general reliability. Among them, the LLM Score aligns most closely with human evaluations, reflecting its strength in capturing the semantic and structural correctness of model diagrams. Overall, metrics incorporating structural and semantic cues correlate better with human judgment than those relying solely on surface-level text similarity.

7 Conclusion

In this paper, we investigate the task of Model Diagram-to-Code Generation. To facilitate research in this area, we construct MDCDataset, a curated dataset comprising 1,008 instances across 16 research domains. This dataset fills a critical gap in current research and provides a benchmark for future studies. Building upon MDCDataset, we conduct a comprehensive study evaluating the performance of MLLMs on this task. The results reveal the limitations of current MLLMs in translating model diagrams into code, highlighting key challenges in visual reasoning and structural understanding. To address these challenges, we propose MDCAgent, a collaborative multi-agent framework specifically designed to analyze complex structural elements and uncover implicit reasoning relationships in model diagrams. Experimental results demonstrate that MDCAgent consistently improves code generation performance across multiple evaluation settings. In summary, this work not only introduces a benchmark dataset but also presents a baseline framework to facilitate future research in this area. Future directions include training specialized vision-language-code models and incorporating domain-specific knowledge to further enhance code generation accuracy and generalization.

Acknowledgments

This research is supported by the National Natural Science Foundation of China (62476097), the Fundamental Research Funds for the Central Universities, South China University of Technology (x2rjD2250190), Guangdong Provincial Fund for Basic and Applied Basic Research—Regional Joint Fund Project (Key Project) (2023B1515120078), Guangdong Provincial Natural Science Foundation for Outstanding Youth Team Project (2024B1515040010).

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Mistral AI. 2023. Announcing Mistral 7B. https://www.mistral.ai/news/announcing-mistral-7b. Accessed: 2024-03-30.

[3] Amazon Web Services. 2024. Amazon Nova Lite v1 Documentation. https://docs.aws.amazon.com/zh_cn/nova/latest/userguide/what-is-nova.html. Accessed: 2025-03-30.

[4] AI Anthropic. 2024. Introducing the next generation of claude.

[5] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923* (2025).

[6] Liuwen Cao, Yi Cai, Jiexin Wang, Hongkui He, and Hailin Huang. 2024. Beyond code: Evaluate thought steps for complex code generation. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. 2296–2306.

[7] Liuwen Cao, Hongkui He, Hailin Huang, Jiexin Wang, and Yi Cai. 2025. Rethinking-based Code Summarization with Chain of Comments. In *Proceedings of the 31st International Conference on Computational Linguistics*. 3043–3056.

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.

[10] Ahana Ghosh, Liina Malva, Alkis Gotovos, Danial Hooshyar, and Adish Singla. 2025. Exploring the Impact of Quizzes Interleaved with Write-Code Tasks in Elementary-Level Visual Programming. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. 381–387.

[11] Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Dan Zhang, Diego Rojas, Guanyu Feng, Hanlin Zhao, et al. 2024. Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793* (2024).

[12] Yucheng Han, Chi Zhang, Xin Chen, Xu Yang, Zhibin Wang, Gang Yu, Bin Fu, and Hanwang Zhang. 2023. Chartllama: A multimodal llm for chart understanding and generation. *arXiv preprint arXiv:2311.16483* (2023).

[13] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. 2024. OlympiadBench: A Challenging Benchmark for Promoting AGI with Olympiad-Level Bilingual Multimodal Scientific Problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3828–3850.

[14] Anwen Hu, Yaya Shi, Haiyang Xu, Jiabo Ye, Qinghao Ye, Ming Yan, Chenliang Li, Qi Qian, Ji Zhang, and Fei Huang. 2024. mplug-paperowl: Scientific diagram analysis with the multimodal large language model. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 6929–6938.

[15] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

[16] ISO/IEC/IEEE. 2011. Systems and software engineering–architecture description. *ISO/IEC/IEEE 42010: 2011 (E)(Revision of ISO/IEC 42010: 2007 and IEEE Std 1471-2000)* 2011 (2011), 1–46.

[17] Yue Jiang, Changkong Zhou, Vikas Garg, and Antti Oulasvirta. 2024. Graph4GUI: Graph Neural Networks for Representing Graphical User Interfaces. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–18.

[18] Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. 2025. Minimax-01: Scaling foundation models with lightning attention. *arXiv preprint arXiv:2501.08313* (2025).

[19] Feng Li, Renrui Zhang, Hao Zhang, Yuanhan Zhang, Bo Li, Wei Li, Zejun Ma, and Chunyuan Li. 2024. Llava-next-interleave: Tackling multi-image, video, and 3d in large multimodal models. *arXiv preprint arXiv:2407.07895* (2024).

[20] Kaixin Li, Yuchen Tian, Qisheng Hu, Ziyang Luo, Zhiyong Huang, and Jing Ma. 2024. MMCode: Benchmarking Multimodal Large Language Models for Code Generation with Visually Rich Programming Problems. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 736–783.

[21] Lei Li, Yuqi Wang, Runxin Xu, Peiyi Wang, Xiachong Feng, Lingpeng Kong, and Qi Liu. 2024. Multimodal ArXiv: A Dataset for Improving Scientific Comprehension of Large Vision-Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 14369–14387.

[22] Yunfan Li, Mouxing Yang, Dezhong Peng, Taihao Li, Jiantao Huang, and Xi Peng. 2022. Twin contrastive learning for online clustering. *International Journal of Computer Vision* 130, 9 (2022), 2205–2221.

[23] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *Advances in neural information processing systems* 36 (2023), 34892–34916.

[24] Haoyu Lu, Wen Liu, Bo Zhang, Bingxuan Wang, Kai Dong, Bo Liu, Jingxiang Sun, Tongzheng Ren, Zhuoshu Li, Hao Yang, et al. 2024. Deepseek-vl: towards real-world vision-language understanding. *arXiv preprint arXiv:2403.05525* (2024).

[25] Meta AI. 2024. Llama 3.2 Model Card and Prompt Formats. https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_2/. Accessed: 2025-03-30.

[26] Victor-Alexandru Pădurean and Adish Singla. [n. d.]. Benchmarking Generative Models on Computational Thinking Tests in Elementary Visual Programming. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

[27] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PmLR, 8748–8763.

[28] Josselin Roberts, Tony Lee, Chi Heem Wong, Michihiro Yasunaga, Yifan Mai, and Percy S Liang. 2025. Image2Struct: Benchmarking Structure Extraction for Vision-Language Models. *Advances in Neural Information Processing Systems* 37 (2025), 115058–115097.

[29] Chufan Shi, Cheng Yang, Yaxin Liu, Bo Shui, Junjie Wang, Mohan Jing, Linran Xu, Xinyu Zhu, Siheng Li, Yuxiang Zhang, et al. 2024. Chartmimic: Evaluating lmm's cross-modal reasoning capability via chart-to-code generation. *arXiv preprint arXiv:2406.09961* (2024).

[30] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2024. Design2code: How far are we from automating front-end engineering? *arXiv e-prints* (2024), arXiv–2403.

[31] Davit Soselia, Khalid Saifullah, and Tianyi Zhou. 2023. Learning UI-to-Code Reverse Generator Using Visual Critic Without Rendering. *arXiv preprint arXiv:2305.14637* (2023).

[32] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[33] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. 2025. Gemma 3 Technical Report. *arXiv preprint arXiv:2503.19786* (2025).

[34] Sverrir Thorgeirsson, Lennart C Lais, Theo B Weidmann, and Zhendong Su. 2024. Recursion in secondary computer science education: A comparative study of visual programming approaches. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 1321–1327.

[35] Birgit Vogel-Heuser, Daniel Schütz, Timo Frank, and Christoph Legat. 2014. Model-driven engineering of manufacturing automation software projects–A SysML-based approach. *Mechatronics* 24, 7 (2014), 883–897.

[36] Yuxuan Wan, Chaozheng Wang, Yi Dong, Wenxuan Wang, Shuqing Li, Yintong Huo, and Michael R Lyu. 2024. Automatically generating UI code from screenshot: A divide-and-conquer-based approach. *arXiv preprint arXiv:2406.16386* (2024).

[37] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. 2024. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191* (2024).

[38] Chengyue Wu, Yixiao Ge, Qiushan Guo, Jiahao Wang, Zhixuan Liang, Zeyu Lu, Ying Shan, and Ping Luo. 2024. Plot2code: A comprehensive benchmark for evaluating multi-modal large language models in code generation from scientific plots. *arXiv preprint arXiv:2405.07990* (2024).

[39] Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Guoyin Wang, Heng Li, Jiangcheng Zhu, Jianqun Chen, et al. 2024. Yi: Open foundation models by 01. ai. *arXiv preprint arXiv:2403.04652* (2024).

[40] Fengji Zhang, Linquan Wu, Huiyu Bai, Guancheng Lin, Xiao Li, Xiao Yu, Yue Wang, Bei Chen, and Jacky Keung. 2024. HumanEval-V: Evaluating Visual Understanding and Reasoning Abilities of Large Multimodal Models Through Coding Tasks. *arXiv preprint arXiv:2410.12381* (2024).

[41] Zhehao Zhang, Weicheng Ma, and Soroush Vosoughi. 2024. Is gpt-4v (ision) all you need for automating academic data visualization? exploring vision-language models' capability in reproducing academic charts. In *Findings of the Association*

for Computational Linguistics: EMNLP 2024. 8271–8288.

[42] Mingjun Zhao, Mengzhen Wang, Yinglong Ma, Di Niu, and Haijiang Wu. 2023. CEIL: A General Classification-Enhanced Iterative Learning Framework for Text Clustering. In *Proceedings of the ACM Web Conference 2023*. 1784–1792.

[43] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5673–5684.